Laboratory report of Nanjing University of Information Science and Technology

Experiment Title Containerization with Docker Data

Class 24 Applied Computing Name Zhou Jiaqi ID 202483910036

一. Experimental Purpose

Learn how to package a web application into a Docker Image.
Understand the concept of Immutable Infrastructure.
Practice mapping network ports between a Cloud Container and the Host.

二. Environment Setup

1. Open your GitHub repository.
2. Click the Code button and select Codespaces.
3. Click Create codespace.

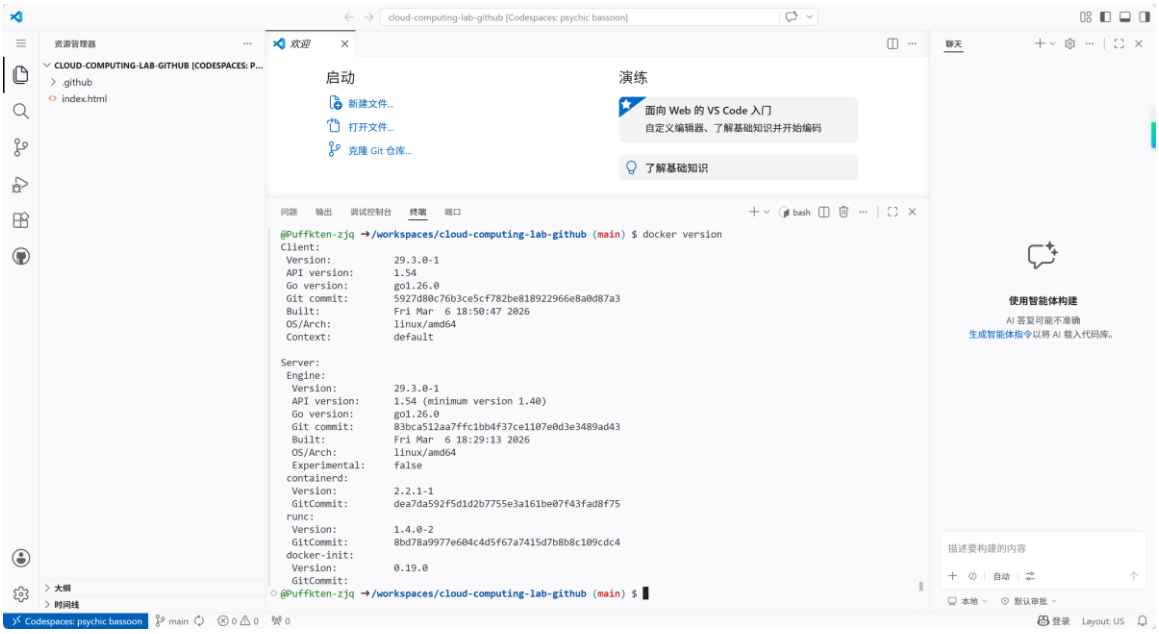

Figure 1: GitHub Codespaces Creation

Step 1: Verify Docker Environment

In your Codespaces terminal, type the following command to ensure the Docker daemon is running:
Bash: docker version
Observation: You should see both Client and Server version information.

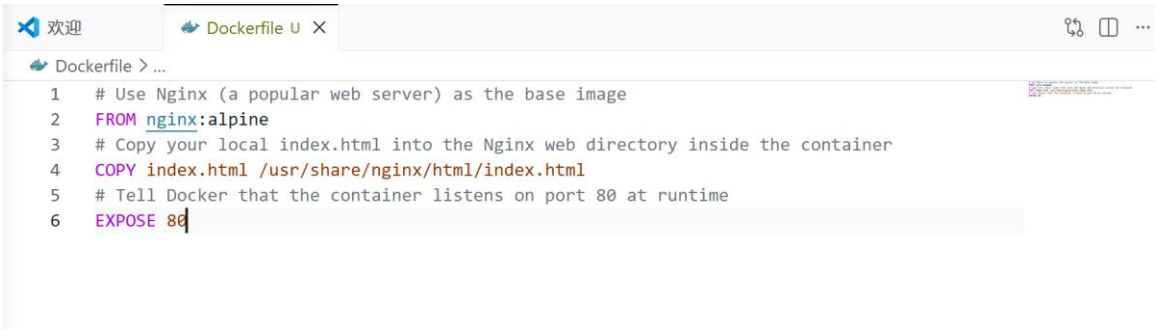

Figure 2: Docker Version Verification

Step 2: Create a Dockerfile

A Dockerfile is a script containing instructions on how to build your image.
Steps:
• In the file explorer on the left, click New File.
• Name it exactly Dockerfile (case-sensitive, no file extension).
• Paste the following code:

```
# Use Nginx as the base image
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

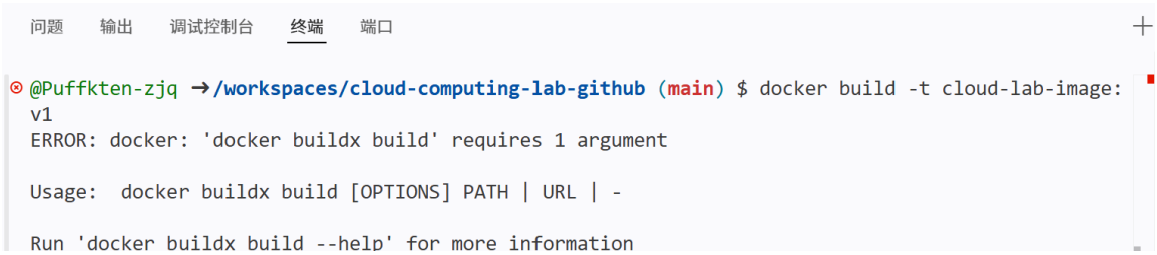

Figure 3: Dockerfile Creation

Step 3: Build the Docker Image

Run the following command to "bake" your code into a reusable image:
Bash: docker build -t cloud-lab-image:v1 .

Cloud Concept: The . at the end tells Docker to look for the Dockerfile in the current folder.

Figure 4: Docker Image Building

Step 4: Run the Container

Now, launch a "Running Instance" (Container) based on your image:
Bash: docker run -d -p 8080:80 --name my-web-container cloud-lab-image:v1
Explanation: -p 8080:80 maps your Cloud VM's port 8080 to the Container's port 80.

Figure 5: Running Docker Container

Step 5: Access the Web Server

Codespaces will detect a new port is active. A pop-up will appear in the bottom right: "A service is running on port 8080."
Click Open in Browser.
Observation: You should see your personal index.html being served from inside a Docker container!

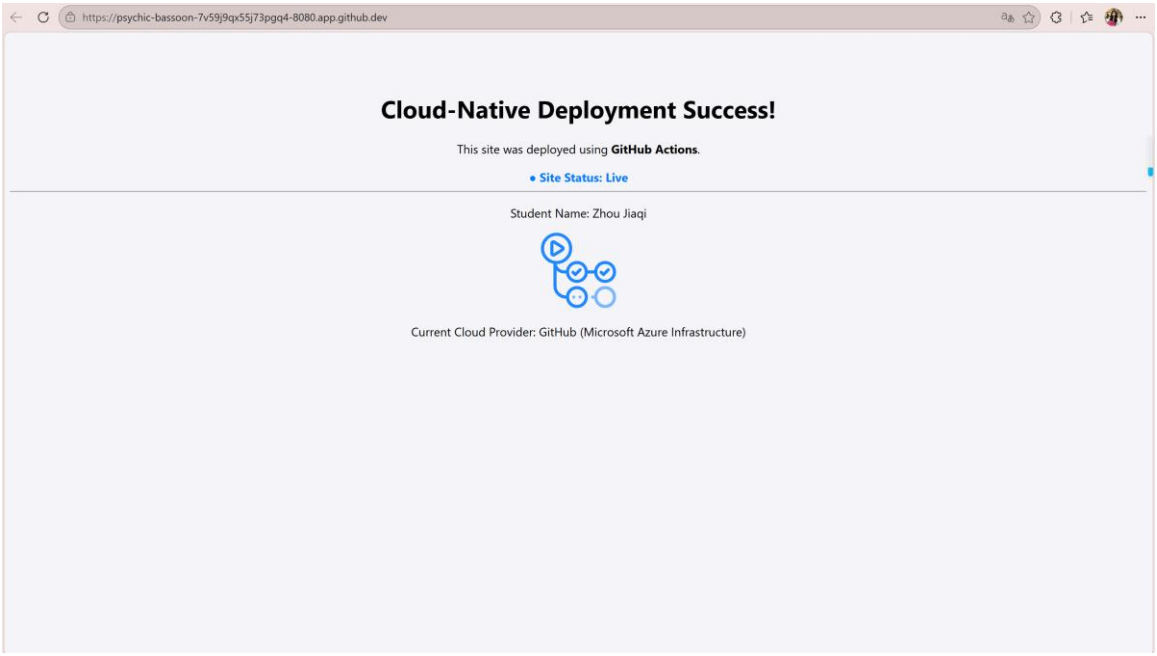

Figure 6: Web Server Access

Step 6: Testing Container Immutability

In Cloud Computing, containers are immutable (unchangeable). This experiment proves why we "build once and run anywhere."

• Modify the Code: Go to your index.html file and change the background color or the text.

• Refresh the Browser: Go back to the tab where your site is running on port 8080 and refresh.

• Observation: The website did not change.

• The Concept: Even though you changed the source file, the Image running inside the container is a static snapshot of the past.

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Cloud Computing Lab </title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            text-align: center;
            padding-top: 50px;
            background-color: #fff3cd;
        }

        .status {
            color: #007bff;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>Cloud-Native Deployment Updated!</h1>
    <p>This site was deployed using <b>GitHub Actions</b>.</p>
    <div class="status">● Site Status: Live</div>
    <hr>
    <p>Student Name: Zhou Jiaqi</p>
    <img src="https://github.githubassets.com/images/modules/site/features/actions-icon-act
    <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
```

Figure 7: Modified index.html File

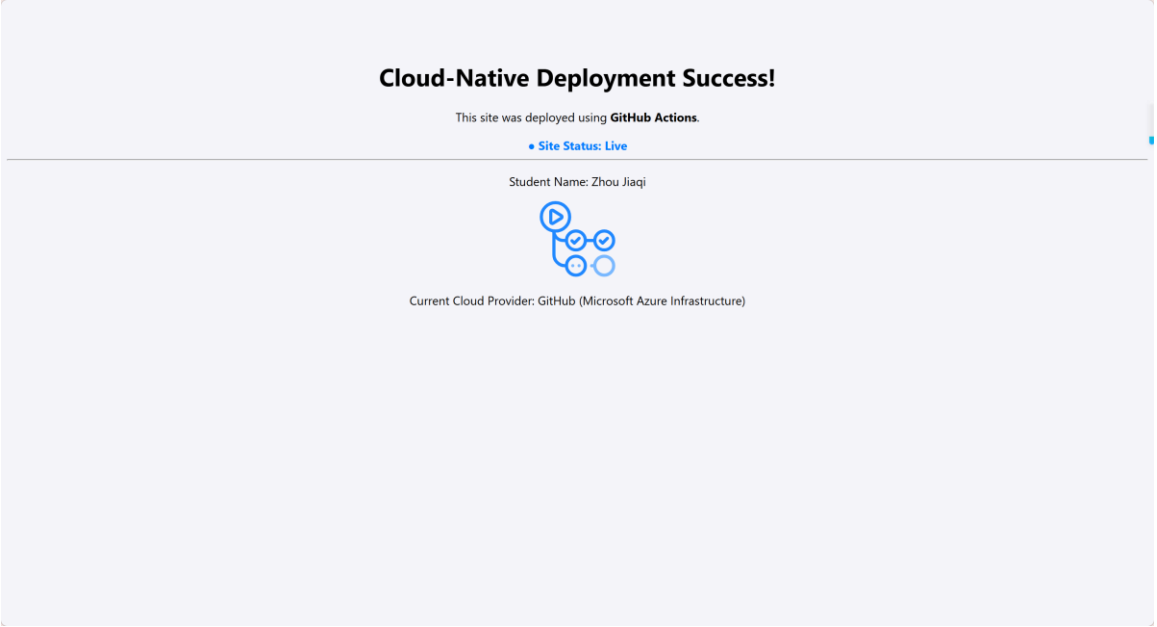

Figure 8: Website Not Updated (Immutability Demonstration)

Step 7: Exploring the Container Interior (Shell Access)

A container is like a mini-computer. Let's "step inside" to see how the cloud organizes files.
Enter the Container:
Bash: docker exec -it my-web-container sh
Navigate Files:
ls /usr/share/nginx/html/

cat /usr/share/nginx/html/index.html

Type exit to return to your main Cloud VM terminal.

The Concept: This demonstrates Isolation. The container has its own file system, separate from your Codespace.

```
@Puffkten-zjq ➜ /workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
<head>
    <meta charset="UTF-8">
    <title>Cloud Computing Lab </title>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            text-align: center;
            padding-top: 50px;
            background-color: #f4f4f9;
        }

        .status {
            color: #007bff;
            font-weight: bold;
        }
    </style>
</head>
<body>
    <h1>Cloud-Native Deployment Success!</h1>
    <p>This site was deployed using <b>GitHub Actions</b>.</p>
    <div class="status">● Site Status: Live</div>
    <hr>
    <p>Student Name: Zhou Jiaqi</p>
    <img src="https://github.githubassets.com/images/modules/site/features/actions-icon-actions.svg" width="100" alt="Actions Logo">
    <p>Current Cloud Provider: GitHub (Microsoft Azure Infrastructure)</p>
</body>
</html>
/ # ^C

/ # exit
@Puffkten-zjq ➜ /workspaces/cloud-computing-lab-github (main) $
```

Figure 9: Container Shell Access

Step 8: Cloud Resource Monitoring

Cloud providers charge based on resource usage. Let's see how "lightweight" containers really are compared to traditional Virtual Machines.

Check Stats:

Bash: docker stats

Analyze: Look at the MEM USAGE and CPU % columns in the real-time table.

Observation: Notice that a full web server is likely using less than 5MB of RAM. A traditional VM would require at least 512MB to 1GB.

The Concept: This is the Efficiency of the cloud. Containers share the host's OS kernel.

```
@Puffkten-zjq ➜ /workspaces/cloud-computing-lab-github (main) $ docker exec -it my-web-container sh
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ # ls /usr/share/nginx/html/
50x.html    index.html
/ # cat /usr/share/nginx/html/index.html
<!DOCTYPE html>
<html lang="en">
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O           BLOCK I/O         PIDS
657b30e51f42   my-web-container   0.00%     3.426MiB / 7.758GiB   0.04%     6.86kB / 4.29kB   20.5kB / 24.6kB   3
```

Figure 10: Docker Resource Monitoring

Step 9: Scaling and Port Management

In the cloud, "Scaling" means running multiple copies of the same service to handle more traffic.
Run a Second Instance:
docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
Verify: Check the "Ports" tab in Codespaces. You should now have two active ports: 8080 and 8081.
Intentional Error: Try running a third container using port 8080 again.
Observation: Docker will throw an error: "Bind for 0.0.0.0:8080 failed: port is already allocated."
The Concept: This is Scalability. We can spin up identical containers instantly.

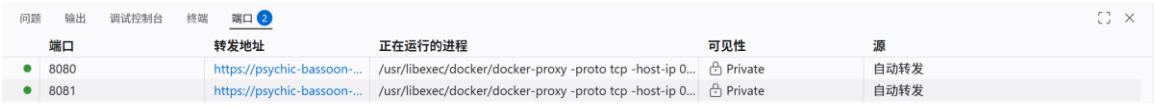

Figure 11: Multiple Containers Running

```
^C@Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) docker run -d -p 8081:80 --name second-container cloud-lab-image:v1
dd454aa1876040d8d568712b58efabf0fc8b21c41bc5114f8e5c50fbe6fff7bb
@Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ docker run -d -p 8080:80 --name third-container cloud-lab-image:v1
58f7c4592761981e4e4a3f6201cc89c4338f328c7bfad67f9a91387f7036eef7
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint t
hird-container (a3ca2474877e1d0f2f80975c6f453ccf4ea18dc1fb2ec8cefae2b2976e86ede2): Bind for 0.0.0.0:8080 failed: port is already allocat
ed

Run 'docker run --help' for more information
```

Figure 12: Port Allocation Error

Step 10: Lab Cleanup

Professional cloud engineers always clean up resources to save costs and keep the environment tidy.
Stop and Remove:
Bash: docker stop my-web-container second-container
Bash: docker rm my-web-container second-container
Verify Cleanup:
Bash: docker ps
The Concept: In a "Pay-as-you-go" cloud model, stopping unused resources prevents unnecessary billing.

```
@Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ docker stop my-web-container second-container docker rm my-web-container
 second-container
my-web-container
second-container
my-web-container
second-container
Error response from daemon: No such container: docker
Error response from daemon: No such container: rm
@Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Figure 13: Container Cleanup

Step 11: Save Changes with Git

• git add . (Gather all your lab changes)
• git commit -m "Finished Lab 02" (Seal the package)
• git push (Ship it to GitHub for permanent storage)

```
● @Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ git add .
● @Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ git commit -m "Finished Lab 02"
  [main deba107] Finished Lab 02
   1 file changed, 6 insertions(+)
   create mode 100644 Dockerfile
● @Puffkten-zjq ➜/workspaces/cloud-computing-lab-github (main) $ git push
  Enumerating objects: 4, done.
  Counting objects: 100% (4/4), done.
  Delta compression using up to 2 threads
  Compressing objects: 100% (3/3), done.
  Writing objects: 100% (3/3), 504 bytes | 504.00 KiB/s, done.
  Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
  To https://github.com/Puffkten-zjq/cloud-computing-lab-github
     2404e77..deba107  main -> main
```

Figure 14: Git Commit and Push

三. Reflection

Problems Encountered and Proposed Solutions

During this experiment, I encountered several challenges:
1. Dockerfile Naming Issue: Initially, I named the file "dockerfile" with a lowercase "d", which caused Docker to fail finding it. Solution: Renamed it to "Dockerfile" with uppercase "D" as required.
2. Port Conflicts: When trying to run multiple containers on the same port, Docker threw an error. Solution: Used different host ports (8080, 8081) for each container instance.
3. Immutability Confusion: At first, I was confused why changes to index.html didn't reflect immediately. Solution: Learned that containers run from static images and require rebuilding.

Why Website Didn't Update Automatically

The website didn't update automatically after editing the code because of the immutable nature of Docker containers. Here's why this is actually a "feature" in cloud computing:

Immutable Infrastructure Principle:
Docker containers are designed to be immutable, meaning once an image is built, it remains unchanged. When you modify the source files on your host machine, the running container continues to use the original image that was created during the build process. This is not a bug but a fundamental design principle.

Why This is a Feature:

1. Consistency: Immutable containers ensure that every instance of your application runs exactly the same code, eliminating "it works on my machine" problems.
2. Reliability: Since containers don't change during runtime, you can reliably deploy the same image across development, testing, and production environments.
3. Reproducibility: If something goes wrong, you can easily roll back to a previous known-good image.
4. Scalability: Immutable images can be quickly replicated across multiple containers, ensuring consistent performance at scale.
5. Security: Immutable infrastructure reduces the attack surface since running containers cannot be accidentally or maliciously modified.